

20-秋-数据结构与算法

Ver. 南京大学软件学院

Powered by *YDJSIR*

基本要求

代码相关

大题Java/C++

小题里面他给的是什么就是什么 (Java优先)

要读得懂代码

期末占比

非常高，请各位同学耗子尾汁，好好复习，不要翻车

基本算法思想

递归

通过重复将问题分解为同类的子问题而解决问题。

出现场景

渗透全课程

贪心（注意和动态规划的区别）

在每一步选择中都采取在当前状态下最好或最优（即最有利）的选择。

出现场景

Prim、Kruskal、Dijkstra

动态规划（注意和贪心的区别）

通过把原问题分解为相对简单的子问题的方式求解复杂问题的方法。

动态规划法试图仅仅解决每个子问题一次，从而减少计算量：一旦某个给定子问题的解已经算出，则将其[记忆化](#)存储，以便下次需要同一个子问题解之时直接查表。

动态规划中不应该有“回溯”，那样就类似于打表了

出现场景

Bellman-Ford

分治

出现场景

归并排序

本文档说明

高亮 表示这是一个算法的名字。

本文档基本上整理自课件，结合了部分在线资源，参考资料将被尽量详细地列出。如果这其中的内容侵犯了您的知识产权，请及时与YDJSIR联系，YDJSIR会尽量及时响应。

第一章 概论

略掉，要求理解，重点在选择题。

$Data\ structure = \{D, R\}; D : DataObject; R : Relationship$

三个方面

数据的逻辑结构

从用户视图看，是面向问题的。

数据的物理结构

从具体实现视图看，是面向计算机的。

数据结构相关的操作及其实现

ADT: Abstract data type——封装

Java & C++ Basics

OO

第二章 算法分析

复杂度计算！（空间 & 时间）

运行时细节（栈空间、堆空间）不考

一般是计算运行次数和给出算法的量级这样的形式。

最好情况、最坏情况、平均情况

算法旁边标注的都是最大复杂度。

O - 上限

Ω - 下限

θ - 上下限同一个量级

顺序查找 - $O(n)$

就硬遍历

秩排序 - $O(n^2)$

```
1 public static void Rank(int[] a, int n, int[] r) {
2     // Rank the n elements a[0:n-1]
3     for (int i = 0; i < n; i++)
4         r[i] = 0;
5     // 这个比较还是很有意思的
6     for (int i = 1; i < n; i++) {
7         for (int j = 0; j < i; j++) {
8             if (a[j] <= a[i])
9                 r[i]++;
10            else
11                r[j]++;
12        }
13    }
14 }
15
16 public static void Rearrange(int[] a, int n, int[] r) {
17     // In-place rearrangement into sorted order
18     for (int i = 0; i < n; i++) {
19         while (r[i] != i) {
20             int t = r[i];
21             swap(a[i], a[t]);
22             swap(r[i], r[t]);
23             // 这个替换的思想还是很有意思的
24         }
25     }
26 }
```

先把每个元素排第几算出来，然后再把每个元素放到该放的位置

选择排序 - $O(n^2)$

直接选择排序

不稳定（数组） 稳定（链表）

```
1 public static void SelectionSort(Comparable[] a){
2     int n = a.length;
3     //sort the n number in a[0:n-1].
4     for(int size=n; size>1; size--){
5         int j=Max(a,size); //找前面size个数里面最大那个
6         swap(a[j],a[size-1]);
7         // put the largest number from the start to the end
8     }
9 }
```

每次都选出最大/最小的元素，然后把它放到前面来

树形选择排序（锦标赛排序）

稳定

堆排序

不稳定

在优先队列章节中有解释

冒泡排序 - $O(n^2)$

稳定

```
1 // 这个就不列了
```

不解释

插入排序 - $O(n^2)$

稳定

```
1 public static void insertionSort( Comparable [ ] a ){
2     int j;
3     for ( int p = 1; p < a.length; p++ ){
4         Comparable tmp = a[ p ];
5         for ( j = p; j > 0 && tmp.compareTo( a[ j -1 ] ) < 0; j-- )
6             a[ j ] = a[ j -1 ];
7         a[ j ] = tmp;
8     }
9 }
```

不解释

他有一个改进，希尔排序，会在最后一章节提到。

折半查找 - $O(\log n)$

```
1 public static int binarySearch(Comparable[] a, Comparable x) {
2     int low = 0, high = a.length - 1;
3     while (low <= high) {
4         int mid = (low + high) / 2;
5         if (a[mid].compareTo(x) < 0)
6             low = mid + 1;
7         else if (a[mid].compareTo(x) > 0)
8             high = mid - 1;
9         else
10            return mid;
11    }
12    return -1;
13 }
14
```

不解释

辗转相除法 - $O(\log n)$

```
1 public static long gcd( long m, long n ){
2     while( n != 0 ){
3         long rem = m % n;
4         m = n;
5         n = rem;
6     }
7     return m;
8 }
```

它的复杂度的计算不做要求

第三和第五章 表 - 线性数据结构

★ 线性表

各种线性表的实现细节与各种操作的复杂度

(单/双) (循环/非循环) 链表

下面的讨论都是基于索引式的链表，但还有一种用数组实现的静态链表值得注意。

大意就是数组里放的值对应于他的下一个元素的位置。

节点

- 数据
- 后指针
- 前指针 (仅双链表)

链表

成员变量

- 头

注意Dummy Head的存在，而且这还一般在题目中是默认的。

- *头、*尾

迭代器

注意链表的一些小技巧，课后习题中便有体现（隐含了归并排序的思想）

双链表不是重点

Trick: 多指针法（YDJSIR自己取的名字）

比如说我要得到一个长为n的序列的第n-p个元素，那么我就在从头开始遍历的时候，在头部指针走过p个元素后再放一个指针进去遍历，当第一个指针走到第n个元素的时候，第二个指针自然走到了第n-p个元素。

关键词：只遍历一次等

约瑟夫环问题

- 循环链表实现
- 数组实现

栈和队列

他们都是特殊的线性表。它们在功能上受到了更多的限制，以适应于特殊用途。

进去一个元素和出去一个元素都是 $O(1)$

栈 - LIFO

`push` `pop` `peek` 等方法，此处不展开。

内部实现可以是数组和链表，但是都被抽象起来了。

栈可以设定容量，所以可能会满。

队列 - FIFO

显然，这玩意和优先队列的实现是根本不一样的！

`inqueue` `enqueue`

队列可以设定容量，所以可能会满。

可以搞循环队列来节约资源。

栈和队列是后面很多算法的基础

基数排序 - $O(\log_B(N) \cdot n)$

PPT说是 $O(d(n + radix))$ d是次数, n是元素数量, radix是桶个数

这里的桶实际上是一个FIFO的队列, 所以放在这个位置上。

补充

- 基数排序: 根据键值的每位数字来分配桶;
- 计数排序: 每个桶只存储单一键值;
- 桶排序: 每个桶存储一定范围的数值;

比如说对一个十进制数, 可以按照这样的顺序来: 先按照个位数进桶, 然后十位, 以此类推, 到最后就排好了。

注意是从低位开始排

哈希表

建表后, 可以实现 $O(1)$ 的对数据的快速查找

重点是取模算法, 其他的反而没那么重要。

散列方法

负载因子与再散列

为了减少冲突处理的次数, 哈希表一般都是稀疏的。我们记整个哈希表容量为 n , 负载因子为 α , 则一旦哈希表内数据量达到 $n\alpha$, 我们就需要扩充表容量了, 这叫再散列。

冲突处理

但最好的方法还是根本就不要产生冲突, 当然考试题显然要制造冲突考验你

开放地址法

线性探查

隐患: 堆积问题导致其他本身没有冲突的 **相邻** 项目也会受到牵连影响

二次探查

隐患: 堆积问题导致其他本身没有冲突的 **相邻** 项目也会受到牵连影响

双重哈希

几乎是最完美的办法。

如果双重哈希还是冲突，那么还是回到线性探查/二次探查/链地址法上

链地址法

冲突了拖个链表就是了，查找的时候需要遍历对应链表

隐患：链表如果拖得太长，效率就会极大降低

第四、第六和第七章 🎄 树 - 非线性数据结构

基本概念与术语

节点、根等

树的高度默认算根节点的和叶节点的，除非题目特别说明。

树里面的度只算出度，所以叶的度为0。

二叉树

满二叉树、完全二叉树等

边和点的关系式

对m阶树而言

- $e = n - 1$
- 第i层最多有 m^i 个元素
- 树中最少有 $h + 1$ 个元素，最多有 $m^h - 1$ 个元素
- $n_0 = n_2 + 1$ ，下标指的是度数
- $n - 1 \geq h \geq \lceil \log_2(n + 1) \rceil - 1$

各种表示二叉树的方式

数组（访问效率高但是不大灵活）

根据索引来定位，这也是堆中的完全二叉树的组织方式

索引

类似链表，较为常用

广义表

不讲

各种遍历的方式

递归

```
1 public void inOrderRec(Node root) {
2     if (root == null) {
3         return;
4     }
5     inOrderTraverse(root.left);
6     System.out.print(root.value+" ");
7     inOrderTraverse(root.right);
8 }
```

按这种方式写就对了，前中后都一样

栈

先序

```
1 public static void preOrder(Node root){
2     if(root == null)return;
3     Stack<Node> s = new Stack<Node>();
4     while(root != null || !s.isEmpty()){
5         while(root != null){
6             System.out.print(root.value + " ");
7             s.push(root); //先访问再入栈
8             root = root.left;
9         }
10        root = s.pop();
11        root = root.right; //如果是null，出栈并处理右子树
12    }
13 }
```

中序

```
1 public static void inOrder(Node root){
2     if(root == null)return;
3     Stack<Node> s = new Stack<Node>();
4     while(root != null || !s.isEmpty()){
5         while(root != null){
6             s.push(root);
7             root = root.left;
8         }
9         root = s.pop();
10        System.out.print(root.value + " ");
11        root = root.right; //如果是null，出栈并处理右子树
12    }
13 }
```

后序

```
1 public void postOrderRec(Node root) {
2     Node pre = null;
3     Stack<Node> stack = new Stack<>();
4     stack.push(root);
5     while (!stack.isEmpty()) {
6         root = stack.peek();
7         if ((root.left == null && root.right == null) || ((pre !=
8 null)&&(root.left == pre || root.right == pre))) {
9             System.out.print(root.value+" ");
10            pre = stack.pop();
11        }else {
12            if (root.right != null) {
13                stack.push(root.right);
14            }
15            if (root.left != null) {
16                stack.push(root.left);
17            }
18        }
19    }
```

按层次遍历-栈

```
1 public static void levelTravel(Node root){
2     if (root == null) return;
3     Queue<Node> q=new LinkedList<Node>();
4     q.add(root);
5     while(!q.isEmpty()){
6         Node temp = q.poll();
7         System.out.println(temp.value);
8         if(temp.left!=null) q.add(temp.left);
9         if(temp.right!=null) q.add(temp.right);
10    }
11 }
```

以上内容的重点在递归算法，非递归算法（基于栈）重在理解。

各种建树的方式

MakeTree

先序 + 中序 / 后序 + 中序

基本思想：用中序找出左子树和右子树，然后分割，递归

先序 $X_1X_2X_3X_4X_5$ **中** $Y_1Y_2Y_3Y_4Y_5$

中序 **中** $X_{i1}X_{i2}X_{i3}X_{i4}X_{i5}$ $Y_{i1}Y_{i2}Y_{i3}Y_{i4}Y_{i5}$

上面的X和Y的顺序虽然不一定一样，但是我们每一步只是切分，直到结果显而易见

先序+后序只在某些特定情况下可以建树

* 广义表

$A(B(D), C(E(\wedge, G), F(H, I)))$

后缀表达式

$(a + b) * c \rightarrow ab + c * \rightarrow$

递归 + 栈

树和森林

表示法

* 广义表

双亲

这种表示方法适用于经常查找父节点的情况（应用：并查集）

★ 左子女右兄弟

将一棵任意的树转换为二叉树

左边放左子女，右边放其兄弟即可（右子女当作左子女的兄弟处理就可以了）

树的遍历

DFS

先根

对应于对应左子女右兄弟二叉树的 **先序**

访问树的根按先序遍历根的第一棵子树，第二棵子树，.....等。

后根

对应于对应左子女右兄弟二叉树的 **中序**

按后序遍历根的第一棵子树，第二棵子树，.....等访问树的根。

BFS

按层次遍历

注意左子女右兄弟特性，对每一迭代都是先一路向右先将所有非空节点纳入一队列中，而后每次出队的时候都必把其所有非空子节点进队即可。

森林的遍历

DFS

先根

对应于左子女右兄弟二叉树的 **先序**

- 访问F的第一棵树的根
- 按先根遍历第一棵树的子树森林
- 按先根遍历其它树组成的森林

中根

对应于左子女右兄弟二叉树的 **中序**

- 按中根遍历第一棵树的子树森林
- 访问F的第一棵树的根
- 按中根遍历其它树组成的森林

后根

对应于左子女右兄弟二叉树的 **后序**

- 按后根遍历第一棵树的子树森林
- 按后根遍历其它树组成的森林
- 访问F的第一棵树的根

BFS

按层次遍历

和树的遍历是一样的。

线段树

节点

left	left thread	data	right thread	right
指针	0 - 左子女 1 - 前驱		0 - 右子女 1 - 后继	右指针

线索树

在建树的时候左右两个数据域都要填上，然后根据实际情况补充 **left thread** 和 **right thread** 的值。

中序线索二叉树

建树的过程与插入/删除后的再平衡过程略（PPT也没有给）

中序线索二叉树的正确食用方式

```
1 // 先找到中序遍历的起点，就是一路向左
2 template<class Type>
3     ThreadNode<Type>* ThreadInorderIterator<Type>::First( ){
4         while (current->leftThread==0) current=current->leftchild;
5         return current;
6     }
7
8 // 找后继
9 template<class Type>
10    ThreadNode<Type>* ThreadInorderIterator<Type>::Next( ){
11        ThreadNode<Type>* p=current->rightchild;
12        if(current->rightThread==0) // 如果该节点有右子女
13            while(p->leftThread==0) p=p->leftchild; // 一路向左找左子女
14        current=p;
15    }
```

增长树

- 对原二叉树中度为1的结点，增加一个空树叶
- 对原二叉树中的树叶，增加两个空树叶

外通路长度（外路径）E：根到每个外结点（增长树的叶子）的路径长度的总和（边数）

内通路长度（内路径）I：根到每个内结点（非叶子）的路径长度的总和（边数）。

结点的带权路径长度：一个结点的权值与结点的路径长度的乘积。

带权的外路径长度：各叶结点的带权路径长度之和。

带权的内路径长度：各非叶结点的带权路径长度之和。

霍夫曼树

是霍夫曼树在数据编码中一种应用。

目的

- 1) 信息的总长度最短。
- 2) 为了译码，任一字符的编码不应是另一字符的编码的前缀

算法：利用Huffman算法，把频率作为外部结点的权，构造具有最小带权外路径长度的扩充二叉树，把每个结点的左子女的边标上0，右子女标上1。这样从根到每个叶子的路径上的号码连接起来，就是外结点的字符编码。

YDJSIR在看到代码之前也迷惑了很久，看了代码立马就明白细节了。

```
1 public Huffman(int a[]) {
2     HuffmanNode parent = null;
3     MinHeap heap;
4
5     // 建立数组a对应的最小堆
6     heap = new MinHeap(a);
7
8     for(int i=0; i<a.length-1; i++) {
9         HuffmanNode left = heap.dumpFromMinimum(); // 最小节点是左孩子
```

```

10     HuffmanNode right = heap.dumpFromMinimum(); // 其次才是右孩子
11
12     // 新建parent节点，左右孩子分别是left和right;
13     // parent的大小是左右孩子之和
14     parent = new HuffmanNode(left.key+right.key, left, right, null);
15     left.parent = parent;
16     right.parent = parent;
17
18     // 将parent节点数据拷贝到"最小堆"中
19     heap.insert(parent);
20 }
21 mRoot = parent; // 整个树的节点
22 }

```

```

1 protected HuffmanNode(int key, HuffmanNode left, HuffmanNode right,
HuffmanNode parent) {
2     this.key = key;
3     this.left = left;
4     this.right = right;
5     this.parent = parent;
6 }

```

字符串

这个想必大家都会了吧

这里提及了怎么用先序和中序合成一棵树

* 广义表

🌟 搜索树！

各种操作

二叉搜索树

一般是左边的key值比根节点小，右边key值的比根节点大，然后就可以在树型结构里面检索节点。

🐟 AVL树！

https://blog.csdn.net/qg_25343557/article/details/89110319

自平衡的二叉搜索树！

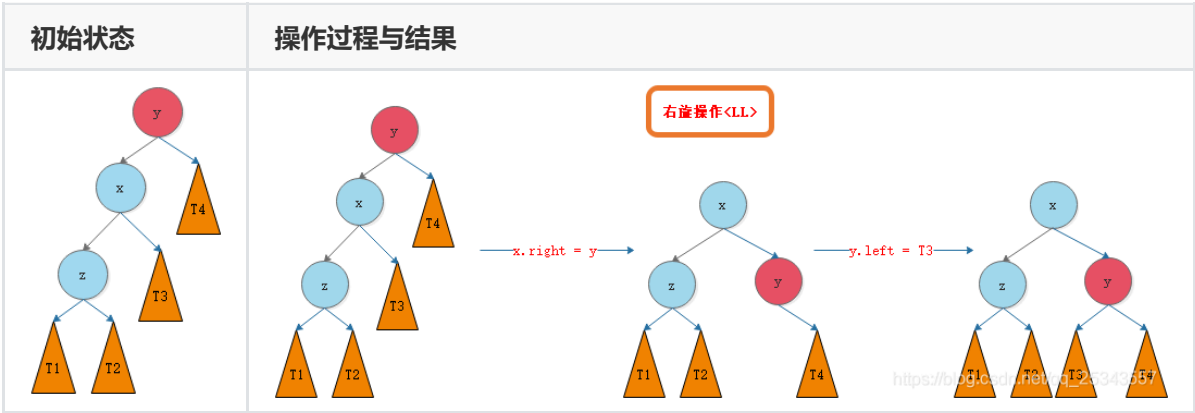
任何时候，左右子树的高度差不超过1。

搜索复杂度： $O(\log n)$ 树高度： $O(\log n)$

下面的左右是指从上往下看树是左树还是右树

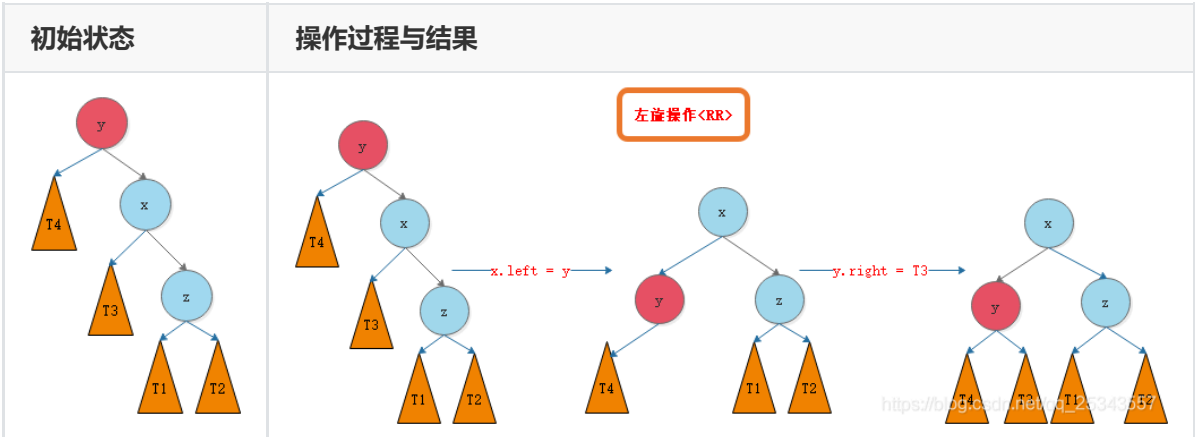
单旋转

LL-右旋



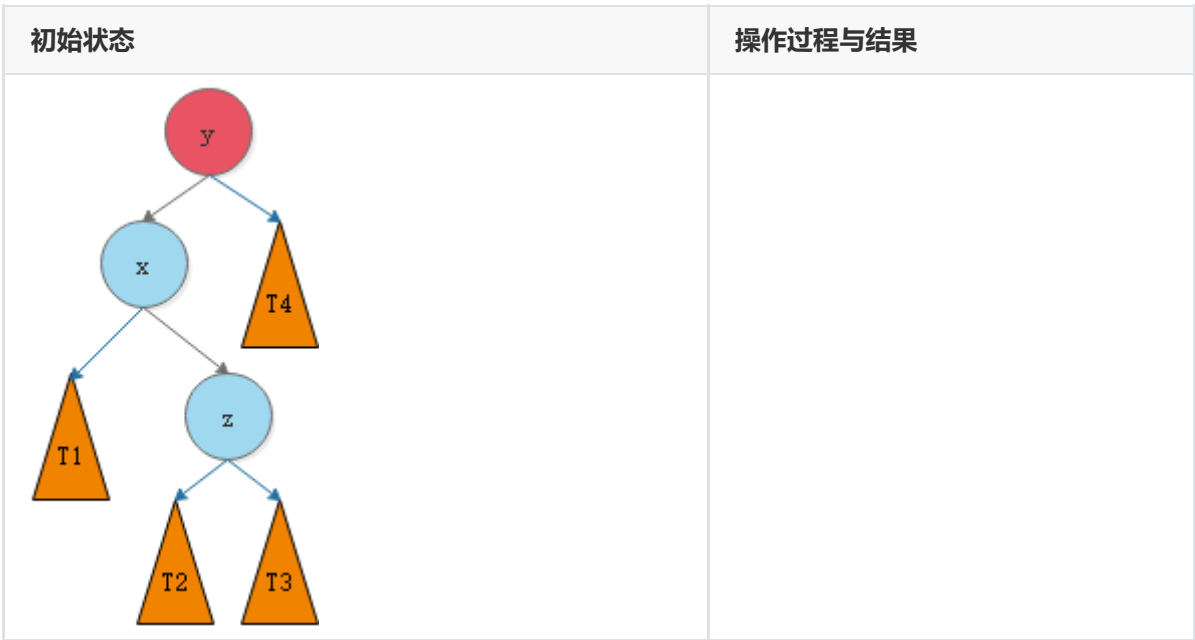
RR-左旋

和上面是一样的，只是中轴对称了一下



双旋转

LR-先左后右



初始状态	操作过程与结果

注意到树的整理应该是由下而上递归进行的！

插入/删除复杂度： $O(\log_2(n))$

证明可以简要看看，用斐波拉契数→斐波拉契树→去掉小量估算

m路搜索树

类似二叉搜索树，但是每个节点里面可能有多个值。

m 路搜索树每个节点里最多有 m-1 个值和 m 个索引。

$$C_0k_1C_1k_2\ldots\ldots k_pC_p$$

B树、* B+树：平衡的m叉搜索树

重点：增/删节点时兄弟节点/父子节点之间的合并

除根节点外，每一个节点里最少有 $\lceil \frac{m}{2} \rceil - 1$ 个分隔值和 $\lceil \frac{m}{2} \rceil$ 个子节点，但所有节点最多只能有 $m - 1$ 个分隔值和 m 个子节点。

最常见的B树应用：磁盘文件索引

最常见的B树：2-3树（三路搜索树）

插入与删除

- B树所有叶节点必须在同一高度。
- 为了维持每一个节点的键值数，按照常规搜索树插入方法插入后需要做调整；

调整方法

自下而上，递归进行

插入

1. 如果节点拥有的元素数量小于最大值，那么有空间容纳新的元素。将新元素插入到这一节点，且保持节点中元素有序。
2. 否则的话这一节点已经满了，将它平均地分裂成两个节点：
 1. 从该节点的原有元素和新的元素中选择出中位数
 2. 小于这一中位数的元素放入左边节点，大于这一中位数的元素放入右边节点，中位数作为分隔值。
 3. 分隔值被插入到父节点中，这可能会造成父节点分裂，分裂父节点时可能又会使它的父节点分裂，以此类推。如果没有父节点（这一节点是根节点），就创建一个新的根节点（增加了树的高度）。

如果分裂一直上升到根节点，那么一个新的根节点会被创建，它有一个分隔值和两个子节点。

删除

定位并删除元素，然后调整树使它满足约束条件；

或者 从上到下处理这棵树，在进入一个节点之前，调整树使得之后一旦遇到了要删除的键，它可以被直接删除而不需要再进行调整（没讲）

借

- 如果缺少元素节点的右兄弟存在且拥有多余的元素，那么向左旋转
 1. 将父节点的分隔值复制到缺少元素节点的最后（分隔值被移下来；缺少元素的节点现在有最小数量的元素）
 2. 将父节点的分隔值替换为右兄弟的第一个元素（右兄弟失去了一个节点但仍然拥有最小数量的元素）
 3. 树又重新平衡
- 否则，如果缺少元素节点的左兄弟存在且拥有多余的元素，那么向右旋转
 1. 将父节点的分隔值复制到缺少元素节点的第一个节点（分隔值被移下来；缺少元素的节点现在有最小数量的元素）
 2. 将父节点的分隔值替换为左兄弟的最后一个元素（左兄弟失去了一个节点但仍然拥有最小数量的元素）
 3. 树又重新平衡

补

- 如果它的两个直接兄弟节点都只有最小数量的元素，那么将它与一个直接兄弟节点以及父节点中它们的分隔值合并
 1. 将分隔值复制到左边的节点（左边的节点可以是缺少元素的节点或者拥有最小数量元素的兄弟节点）
 2. 将右边节点中所有的元素移动到左边节点（左边节点现在拥有最大数量的元素，右边节点为空）
 3. 将父节点中的分隔值和空的右子树移除（父节点失去了一个元素）
 - 如果父节点是根节点并且没有元素了，那么释放它并且让合并之后的节点成为新的根节点（树的深度减小）
 - 否则，如果父节点的元素数量小于最小值，重新平衡父节点

优先队列 - 堆

只要求父节点key值大于/小于子节点key值的完全二叉树。

最小堆是顶上是最小值，最大堆是顶上是最大值。

找父节点下标

下标	0	1	2	3	4	5	6	7	8	9
父节点	-1	0	0	1	1	2	2	3	3	4
序数	1	2	3	4	5	6	7	8	9	10

下标表示 - $\lfloor \frac{i-1}{2} \rfloor$

建堆

下面以最大堆为例

自下而上：元素是不断进来的，不断插入并调整堆 - $O(n)$

插入 - $O(\log_2 n)$

```
1  template<class T>MaxHeap<T>::Insert(const T& x){
2      if(CurrentSize==MaxSize)throw NoMem();
3      int i=++CurrentSize;
4      while(i!=1&&x>heap[i/2]){ // 如果这个元素比父节点大就往上
5          heap[i]=heap[i/2]; i/=2;
6      }
7      heap[i]=x;
8      return *this;
9  }
```

重复 n 次，但是并不是简单乘起来，因为事实上每次都可以节省很多工作量，有证明过程

自上而下：一开始就有所有的元素，直接通过调整得到堆 - $O(\log_2 n)$

```
1  template<class T> void MaxHeap<T>::Initialize (T a[],int size,int ArraySize)
2  {
3      delete[] heap;
4      heap=a;
5      CurrentSize=size;
6      MaxSize=ArraySize;
7      for( int i=CurrentSize/2; i>=1; i--){ // 注意一下是从最后开始
8          T y=heap[i];
9          int c=2*i;
10         while(c<=CurrentSize){
11             if(c<CurrentSize && heap[c]<heap[c+1]) c++;
12             if(y>=heap[c]) break;
13             heap[c/2] = heap[c];
14             c*=2;
15         }
16         heap[c/2]=y;
17     }
```

```
16 | }  
17 | }
```

优先队列

一个一直维护着的堆，按照优先级大小出队。

堆排序

先建好一个优先队列（堆），然后一个个出列，就这么简单。

并查集

重点在使用，一般不单独考察内部具体实现。

主要操作

合并两个不相交集合(并)

判断两个元素是否属于同一集合(查)

并查集的优化

路径压缩

按秩合并

时间及空间复杂度

时间复杂度：可以控制在 $O(1)$

空间复杂度： $O(n)$

第八章 🌟 图 - 非线性数据结构

概念理解

各种图、完全图、简单图、子图、路径、简单通路等

有向图和无向图

很多时候会造成很多区别

度

有向图

入度、出度与度

$$TD(v) = ID(v) + OD(v)$$

无向图

度就是与这一顶点相连的边的数量

连通图/连通分量

强连通与弱联通

- 无向图只有强连通和非联通
- 有向图中如果对任意两顶点 i, j 都有从 i 到 j 与从 j 到 i 的通路, 那么他是强连通的;
- 有向图中如果对任意两顶点 i, j 都有从 i 到 j 或从 j 到 i 的通路, 那么他是弱连通的;

图的表示

邻接矩阵 - 适合高密度的矩阵 (节约空间) - 空间复杂度 $O(n^2)$

无向图下这是一个对称的矩阵。因此你可以只用考虑半边。

邻接表 - 适合稀疏矩阵 (使用链表)

n 个顶点 e 条边的无向图的邻接表表示中有 n 个顶点表结点和 $2e$ 个边表结点。(换句话说, 每条边 (i, j) 在邻接表 中出现两次: 一次在关于 i 的邻接表中, 另一次在关于 j 的邻接表中)

注意 `getFirstNeighbour` 和 `getNextNeighbour` 方法。

逆连接表

从记出边到计入边

邻接多重表(只是带过去没有细讲)

在无向图中, 如果边数为 m , 则在邻接表表示中需 $2m$ 个单位来存储. 为了克服这一缺点, 采用邻接多重表, 每条边用一个结点表示。

图的相关算法

图的遍历

DFS

邻接矩阵

略

邻接表

```
1  template<NameType,DistType> void Graph<NameType,DistType> :: DFS( ){
2      int *visited=new int[NumVertices];
3      for ( int i=0; i<NumVertices; i++) visited[i]=0;DFS(0,visited); //从顶点0
    开始深度优先搜索delete[] visited;
4  }
5
6  template<NameType,DistType> void Graph<NameType,DistType> :: DFS(int v,
    visited[]){
7      cout<<GetValue(v)<<“ ”;
```

```

8     visited[v]=1;
9     int w=GetFirstNeighbor(v);
10    while (w!=-1){
11        if(!visited[w]) DFS(w,visited);
12        w=GetNextNeighbor(v,w);
13    }
14 }

```

BFS

邻接矩阵

略

邻接表

```

1  template<NameType,DistType> void Graph<NameType,DistType> :: BFS(int v){
2      int* visited=new int[NumVertices];
3      for (int i=0; i<NumVertices; i++) visited[i]=0;
4      cout<<GetValue(v)<<'\n';
5      visited[v]=1;queue<int> q;
6      q.Enqueue(v);
7      while(!q.IsEmpty()){
8          v=q.DeQueue();
9          int w=GetFirstNeighbor(v);
10         while (w!=-1){
11             if(!visited[w]){
12                 cout<<GetValue(w)<<'\n';
13                 visited[w]=1;q.Enqueue(w);
14             }
15             w=GetNextNeighbor(v,w);
16         }
17     }
18     delete[] visited;
19 }

```

邻接表下BFS和DFS都是 $O(n + e)$

邻接矩阵下BFS和DFS都是 $O(n^2)$

最小生成树

设 $G = (V, E)$ 是一个连通的无向图 (或是强连通有向图) ,

从图G中的任一顶点出发作遍历图的操作, 把遍历走过的边的集合记为 $TE(G)$, 显然 $G' = (V, TE)$ 是G之子图, G'被称为G的生成树 (spanning tree) 。

生成树是不唯一的, 但一定有 $n - 1$ 条边。

Kruskal - 基于边 - $O(|E|\log|V|)$

PPT上面的示范代码的确是 $O(e\log_2 n + n^2)$, 由于e最大可以取到 $\frac{n(n-1)}{2}$, 达到了 $O(n^2)$ 量级, 故 $e\log_2 e$ 可以做到比 n^2 量级大, 但也可能因为e比较小而导致它的量级更小。

先把边按权值从小到大排 (建堆), 然后依次往结果边集里面加, 每加一次都要检查是否成环 (并查集), 加到 $n - 1$ 条边即可。

```

1 void Graph<string , float>::Kruskal(MinSpanTree&T){
2     MSTEdgeNode e;
3     MinHeap<MSTEdgeNode>H(currentEdges);
4     int NumVertices=VerticesList.Last , u , v;
5     Ufsets F(NumVertices); //建立n个单元素的连通分量for(u=0;u<NumVertices;u++)
6     for (v=u+1;v<NumVertices;v++)if(Edge[u][v]!=MAXINT){
7         e.tail=u;
8         e.head=v;
9         e.cost=Edge[u][v];
10        H.insert(e);}
11    int count=1; //生成树边计数
12    while(count<NumVertices){
13        H.RemoveMin(e);u=F.Find(e.tail);// 防止成环
14        v=F.Find(e.head);
15        if(u!=v){
16            F.union(u,v);
17            T.Insert(e);
18            count++;
19        }
20    }
21 }

```

Prim - 基于点 - $O(n^2)$

任取一个边开始，每次都找已选点集 u 到未选点集 v 之间的最短边加入边集中，并将该边另一顶点加入已选点集中，直至已加入所有点集。

复杂度用连接表不优化可能要 $O(n^3)$ 复杂度。

最短路径

设 $G = (V, E)$ 是一个带权图（有向，无向），如果从顶点 v 到顶点 w 的一条路径为 $(v, v_1, v_2, \dots, w)$ ，其路径长度不大于从 v 到 w 的所有其它路径的长度，则该路径为从 v 到 w 的最短路径。

Dijkstra - $O(n^2)$ - 无负权值单源最短路径

关键：贪心思想

```

1 /**
2     path数组记录从源节点这一节点到目标结点路径上的倒数第二个节点，以此类推
3     dist数组记录到该点的当前已知最短路径长度
4     s记录该点是否已经加入已选点集中
5     图用邻接矩阵表示
6 */
7 public int[] shortestpath(int v){
8     int n = this.numOfVertex;
9     // 初始化
10    for( int i=0; i<n; i++){
11        dist[i]=Edge[v][i];
12        s[i]=0;
13        if( i!=v && dist[i]< MAXNUM ) path[i]=v;
14        else path[i]=-1;
15    }
16    s[v]=1;
17    dist[v]=0;

```

```

18     for( i=0; i<n-1; i++){
19         float min=MAXNUM;
20         int u = v;
21         // 找出最近的点集
22         for( int j=0; j<n; j++)
23             if( !s[j] && dist[j]<min ) {
24                 u=j;
25                 min=dist[j];
26             }
27         s[u]=1; // 把这个最近的点纳入已知点集中
28         // 更新距离
29         for ( int w=0; w<n; w++)
30             if( !s[w] && Edge[u][w] < MAXNUM && dist[u] + Edge[u][w] <
dist[w]){
31                 // 只需要考虑未被加入点集中的点
32                 dist[w]=dist[u]+Edge[u][w];
33                 path[w]=u;
34             }
35     }
36     return path;
37 }

```

按最短路径长度递增的次序产生最短路径。

Bellman-Ford - $O(n^3)$ - 无负环

关键：动态规划

递推公式（动态规划所需要的）

$$dist^1[u] = Edge[v][u]$$

$$dist^k[u] = \min\{dist^{k-1}[u], \min\{dist^{k-1}[j] + Edge[j][u]\}, j = 0, 1, 2, \dots, n-1\}$$

```

1  public int[] BellmanFord(int v){
2      int n = this.numOfVertex;
3      for(int i=0; i<n; i++){
4          dist[i]=Edge[v][i];
5          if(i!=v&&dist[i]<MAXNUM)
6              path[i]=v;
7          else path[i]=-1;
8      }
9      for (int k=2; k<n; k++) // 事实上可以不做满这么多次，当你发现距离数组已经不变化的时
候就可以不做了
10         for(int u=0; u<n; u++)
11             if(u!=v)
12                 for(i=0; i<n; i++)
13                     if (Edge[i][u] < >0 && Edge[i][u]<MAXNUM &&
dist[u]>dist[i]+Edge[i][u]){
14                         dist[u]=dist[i]+Edge[i][u];
15                         path[u]=i;
16                     }
17     return path;
18 }

```

和Dijkstra的一个鲜明区别是它不用考虑什么加不加入点集，他直接就把所有的距离都更新一次，保留了更多的结果。也正因如此，它能够解决负权值问题。

Floyd - $O(n^3)$ - 无负权值 - 所有点到所有点的最短路径

所有顶点之间的最短路径 (Floyd)前提: 各边权值均大于0的带权有向图。

1. 把有向图的每一个顶点作为源点，重复执行Dijkstra算法 n 次，执行时间为 $O(n^3)$
2. Floyd方法，算法形式更简单些，但是时间仍然是 $O(n^3)$

回忆离散课的内容

```
1 public int[][] Alllength(int n){
2     for(int i=0; i<n; i++){
3         for(int j=0; j<n; j++){
4             a[i][j]=Edge[i][j];
5             if(i==j) a[i][j]=0;
6             if(i<>j&& a[i][j]<MAXNUM)
7                 path[i][j]=i;
8             else path[i][j]=0;
9         }
10    }
11    for(int k=0; k<n; k++){
12        for(int i=0; i<n; i++){
13            for(int j=0; j<n; j++){
14                if( a[i][k]+a[k][j]<a[i][j] ){
15                    a[i][j]=a[i][k]+a[k][j];
16                    path[i][j]=path[k][j];
17                }
18            }
19        }
20    }
21    return path;
22 }
```

找出有向图中所有的环

```
1 public class GetAllCyclesForDirectedGraph{
2     static List<Integer> trace; // 当前搜索路径
3     static Set<Integer> searched = new HashSet<>(); // 已被搜索过的节点
4     static Set<List<Integer>> allCircles = new HashSet<>(); // 结果的收集列表
5
6     public static void main(String[] args) {
7         int[][] e = { // 此处的图用邻接矩阵表示
8             {0,1,1,0,0,0,0},
9             {0,0,0,1,0,0,0},
10            {0,0,0,0,0,1,0},
11            {0,0,0,0,1,0,0},
12            {0,0,1,0,0,0,0},
13            {0,0,0,0,1,0,1},
14            {1,0,1,0,0,0,0}};
15        findCircles(e);
16    }
17
18    public static void findCircles(int[][] e){
19        int n = e.length;
20        for(int i = 0; i < n; i++){
```



```

21 // 如果一个节点已经被搜索过一次，那么即使他出现在另一个环中，它也必然在搜索另
    外那个环中其他节点的时候被提及，这一步事实上完成了去重
22     if(searched.contains(i))
23         continue;
24     trace = new ArrayList<>();
25     findCycle(i,e);
26 }
27
28 if(allCircles.size() == 0){
29     System.out.println("There aren't any CIRCLES in the graph!");
30 }
31 int cnt = 1;
32 for(List<Integer> list:allCircles) {
33     System.out.println("CIRCLE" + cnt + ": " + list);
34     cnt++;
35 }
36 }
37
38 /**
39  * 利用DFS算法，从节点v开始寻路
40  * @param v
41  * @param e
42  */
43 private static void findCycle(int v, int[][]e){
44     int j = trace.indexOf(v);
45     if( j != -1) {
46         List<Integer> circle = new ArrayList<>();
47         while(j < trace.size()) {
48             circle.add(trace.get(j));
49             j++;
50         }
51         Collections.sort(circle); // 找出一条环路，对路径内节点进行排序
52         allCircles.add(circle);
53         return;
54     }
55
56     trace.add(v);
57     for(int i = 0; i < e.length; i++) {
58         if(e[v][i] == 1){
59             searched.add(i);
60             findCycle(i,e); // 递归查找
61         }
62     }
63     trace.remove(trace.size() - 1); // 回溯
64 }
65 }

```

找出无向图中所有的环

参考Kruskal中的代码。

图的应用

拓扑排序 - $O(n + e)$

拓扑排序可以检测环路是否存在（但不能给出具体的环路）。拓扑排序不唯一。

搞一个队列，每次都把图中入度为0的节点加入队列中而后在图中删去这些点的所有出边直到表中没有顶点为止。如果出现了图中还有顶点但是无入度为0的点的情况，说明图中有环路。算法用到了队列（栈也可以）可以复习下静态链表（数组实现的链表）

```
1 public void topSort( ) throws CycleFound{
2     Queue q = new Queue();
3     int counter = 0;
4     Vertex v, w;
5     q = new Queue( );
6     for (Vertex v:NodeTable)
7         if( v.indegree == 0 )
8             q.enqueue( v );
9     while( !q.isEmpty( ) ){
10        v = q.dequeue( );
11        v.topNum = ++counter; //Assign next number
12        for(Edge e = v.adj; e != null; e = e.next){
13            w = e.dest;
14            if( --w.indegree == 0 ) q.enqueue(w);
15        }
16    }
17    if( counter != NUM_VERTICES )
18        throw new CycleFound( );
19 }
```

AOV - 无环有向图 - $O(n^2)$

用顶点表示事件，可用来表示事件发生顺序。一个拓扑排序的事情。

AOE - 无环有向图 - $O(n + e)$

顶点表示事件，有向边表示活动，有向边的权值代表完成该项活动所需时间。

- 有唯一的“入度”为0的开始结点
- 唯一的“出度：为0的完成结点

可以由此得出 关键活动 最早发生时间 最迟发生时间 等。

关键路径

1. 目的: 利用事件顶点网络，研究完成整个工程需要多少时间加快那些活动的速度后，可使整个工程提前完成。
2. 关键路径：具有从开始顶点(源点) →完成顶点（汇点）的最长的路径

对事件而言

$Ve[i]$ - 表示事件 V_i 的可能最早发生时间，定义为从源点 $V_0 \rightarrow V_i$ 的最长路径长度，（得满足前置条件）

$Vl[i]$ - 表示事件 V_i 的允许的最晚发生时间。是在保证汇点 V_{n-1} 在 $Ve[n-1]$ 完成的前提下，事件 V_i 允许发生的最晚时间，即 $Ve[n-1] - V_i \rightarrow V_{n-1}$ 的最长路径长度。（不能拖后腿）

对活动而言

$e[k]$ - 表示活动 $a_k = \langle V_i, V_j \rangle$ 的可能的最早开始时间。即等于事件 V_i 的可能最早发生时间。

$$e[k] = Ve[i]$$

$l[k]$ - 表示活动 $a_k = \langle V_i, V_j \rangle$ 的允许的最迟开始时间

$$l[k] = Vl[j] - dur(\langle i, j \rangle);$$

$s[k]$ - 表示活动 a_k 的最早可能开始时间和最迟

$$s[k] = l[k] - e[k]$$

$s[k] = 0$ 表示这是一个关键活动。

以上的计算必须在拓扑有序及逆拓扑有序的前提下进行。求 $Ve[i]$ 必须使 V_i 的所有前驱结点的 Ve 都求得
求 $Vl[i]$ 必须使 V_i 的所有后继结点最晚发生时间都求得。

第九章 排序

排序： n 个对象的序列 $R[0], R[1], R[2], \dots, R[n-1]$ ，按其 **关键码** 的大小，进行由小到大（非递减）或由大到小（非递增）的次序重新排列。

Java的 `Comparable` 接口和C++的运算符重载

内排序：对内存中的 n 个对象进行排序。

外排序：内存放不下，还要使用外存的排序。

二分法插入排序 - $O(n^2)$

稳定

略，但还是比直接插入排序快

希尔排序 - 不定

缩小增量排序（diminishing - increament sort）

- 1) 取一增量（间隔 $gap < n$ ），按增量分组，对每组使用**直接插入排序**或**其他方法**进行排序。
- 2) 减少增量（分的组减少，但每组记录增多）。直至增量为1，即为一个组时。

关键：增量的选取

不稳定

关键思想：分组排序

这是极为困难的问题，因而希尔排序的复杂度有很大变数。目前也有一些较好的解决方案。

Donald Shell最初建议步长选择为 $\frac{n}{2}$ 并且对步长取半直到步长达到1。虽然这样取可以比 $O(n^2)$ 类的算法（插入排序）更好，但这样仍然有减少平均时间和最差时间的余地。

其他方案略，题目里应该会给你特定序列

```
1 public static void shellsort( Comparable [ ] a ){
2     for ( int gap = a.length / 2; gap > 0; gap /= 2 )
3         for ( int i = gap; i < a.length; i++ ){
4             Comparable tmp = a[ i ];
5             int j = i;
6             for ( ; j >= gap && tmp.compareTo( a[ j-gap ] ) < 0; j -= gap )
7                 a[ j ] = a[ j -gap ];
8             a[ j ] = tmp;
9         }
10 }
```

表插入排序 - $O(n^2)$

把插入排序的载体改为链表即可，无需移动数据只用改动指针。

快速排序 - $O(n\log_2 n)$

下面以从小到大排序为例

- 1) 在n个对象中，取一个对象（如第一个对象——基准pivot），按该对象的关键码把所有 $\leq$ 该关键码的对象分划在它的左边。 $\geq$ 该关键码的对象分划在它的右边。
- 2) 对左边和右边（子序列）分别再用快排序。

```
1 template <class Type> void QuickSort( datalist <Type>& list, const int left,
2     const int right ){
3     if (left<right){
4         int pivotpos=partition (list, left, right);
5         QuickSort(list, left, pivotpos-1);
6         QuickSort(list, pivotpos+1, right);
7     }
8 }
```

关键：对基准的选取

方法0：选取第一个数（效果不好）

方法1：随机选取pivot, 但随机数的生成一般是昂贵的。

方法2：三数中值分割法（Median-of-Three partitioning）

N个数，最好选第 $\lceil N/2 \rceil$ 个最大数，这是最好的中值，但这是很困难的。

一般选左端、右端和中心位置上的三个元素的中值作为枢纽元。

8, 1, 4, 9, 6, 3, 5, 2, 7, 0 (8, 6, 0)

具体实现时：将8, 6, 0先排序，即0, 1, 4, 9, 6, 3, 5, 2, 7, 8得到中值pivot为6。

分割策略

将pivot与最后倒数第二个元素交换，使得pivot离开要被分割的数据段。然后，i 指向第一个元素，j 指向倒数第二个元素。0, 1, 4, 9, 7, 3, 5, 2, 6, 8 然后进行分划。

```
1 private static void quicksort( Comparable [ ] a, int left, int right ){
2     if( left + CUTOFF <= right ){ // 如果数量太少用快速排序不划算，所以下面用的是插入排序
3         Comparable pivot = median3( a, left, right );
4         int i = left;
5         int j = right -1;
6         for( ; ; ){
7             while( a[ ++i ] . compareTo( pivot ) < 0 ) { }
8             while( a[ --j ] . compareTo( pivot ) > 0 ) { }
9             if( i < j ) swapReferences( a, i, j );
10            else break;
11        }
12        swapReferences( a, i, right -1 ); // 前面的方法会暂时把基准元素放到最后所以这一步要纠正
13        quicksort( a, left, i -1 );
14        quicksort( a, i + 1, right );
15    }
16    else
17        insertionSort( a, left, right );
18 }
```

归并排序 - $O(n\log_2 n)$

关键思想：分组排序而后归并、分治

归并：两个（多个）有序的文件组合成一个有序文件

以两个序列为例

```
1 template<class Type> void merge(datalist<Type> & initList, datalist<Type> &
mergedList, const int l, const int m, const int n){
2     int i=l, j=m+1, k=1;
3     while ( i<=m && j<=n )
4         if (initList.Vector[i].getKey()<initList.Vector[j].getKey()){
5             mergedList.Vector[k]=initList.Vector[i]; i++;k++;}
6
7     else {mergedList.Vector[k]=initList.Vector[j]; j++; k++;}
8
9     if ( i<=m)
10        for (int n1=k, n2=i; n1<=n && n2<=m; n1++, n2++)
11            mergedList.Vector[n1]=initList.Vector[n2];
12    else
13        for (int n1=k, n2=j; n1<=n && n2<=n; n1++, n2++)
14            mergedList.Vector[n1]=initList.Vector[n2];
15 }
```

方法：每次取出两个序列中的小的元素输出之；当一序列完，则输出另一序列的剩余部分。

迭代的归并排序

如果是对链表操作，分割链表需要改为切断链表（指针置为空），然后合并的时候再连起来

总结

- 注意到二分插入排序和直接选择排序的**比较次数**是固定的；
- 快速排序在已经排好序的情况下反而需要 $\frac{n*(n-1)}{2}$ 次比较；
- 直接选择排序的移动次数耶斯固定的；
- 归并和表插入排序的辅助存储为 $O(n)$ 量级，快排是 $O(\log_2 n)$ 量级